



INSTITUTO POLITÉCNICO NACIONAL



Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

PRACTICA 4

PROBLEMAS DE CONCURRENCIA

Alumnos:

Martínez Barrueta Mariana
2024640160

Gámez Gress Humberto Isaac
2024640024

Zuñiga Aviles Nathan Emiliano
2024640103

Base De Datos Distribuidos

Caso 1: Uncommitted Dependency (Dirty Read)

La **Lectura Sucia** (*Dirty Read* o *Uncommitted Dependency*) ocurre cuando una transacción concurrente lee datos que han sido modificados por otra transacción activa pero que **aún no han sido confirmados** (COMMIT). Si la transacción que modificó los datos ejecuta posteriormente una reversión (ROLLBACK), los datos leídos por la segunda transacción quedan invalidados y dejan de existir formalmente en el registro de la base de datos, violando la integridad y el principio de aislamiento transaccional.

- **Ocorre en:** READ UNCOMMITTED (o cuando se utiliza la sugerencia de consulta WITH (NOLOCK)), donde el motor ignora los bloqueos exclusivos de escritura y no solicita bloqueos compartidos de lectura.
- **Se previene en:** READ COMMITTED (nivel por defecto en SQL Server), REPEATABLE READ y SERIALIZABLE. En estos niveles, una consulta no puede leer filas modificadas hasta que la transacción propietaria libere sus bloqueos exclusivos mediante

Para simular este escenario en la bdd **AdventureWorks**, utilizaremos la tabla Production.Product y actualizaremos temporalmente el campo ListPrice de un artículo de muestra (ProductID = 707).

Vamos a abrir 2 sesiones para simular que son dos equipos los que crearan una sesión de consulta

Sesion 1:

```
SQLQuery1.s...\\Isaac (56))* X
1  USE AdventureWorks2022;
2  GO
3
4  -- Verificamos el valor inicial de control
5  SELECT ProductID, Name, ListPrice
6  FROM Production.Product
7  WHERE ProductID = 707;
8
9  BEGIN TRANSACTION;
10
11  -- Modificamos el precio incrementándolo un 20%
12  UPDATE Production.Product
13  SET ListPrice = ListPrice * 1.20
14  WHERE ProductID = 707;
15
16  -- Simulación de procesamiento transaccional en curso (15 segundos)
17  WAITFOR DELAY '00:00:15';
18
19  -- Revertimos los cambios como si hubiese fallado una regla de negocio
20  ROLLBACK TRANSACTION;
21
22  -- Confirmamos que el precio volvió a su estado original
23  SELECT ProductID, Name, ListPrice
24  FROM Production.Product
25  WHERE ProductID = 707;
```

El valor inicial ListPrice del ProductID 707 es 34.99

	ProductID	Name	ListPrice
1	707	Sport-100 Helmet, Red	34.99

Sesión 2:

```
SQLQuery1.sq...\Isaac (56))* SQLQuery2.s...\Isaac (65))*
1 USE AdventureWorks;
2 GO
3
4 -- Establecemos el nivel permisivo para forzar la lectura sin bloqueos
5 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
6
7 -- Consulta ejecutada mientras la Sesión 1 mantiene la modificación sin confirmar
8 SELECT ProductID, Name, ListPrice
9 FROM Production.Product
10 WHERE ProductID = 707;
11
12 -- Restauramos el nivel por defecto de la sesión
13 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Al ejecutar la transacción en ambas sesiones a la vez:

Sesión 1:

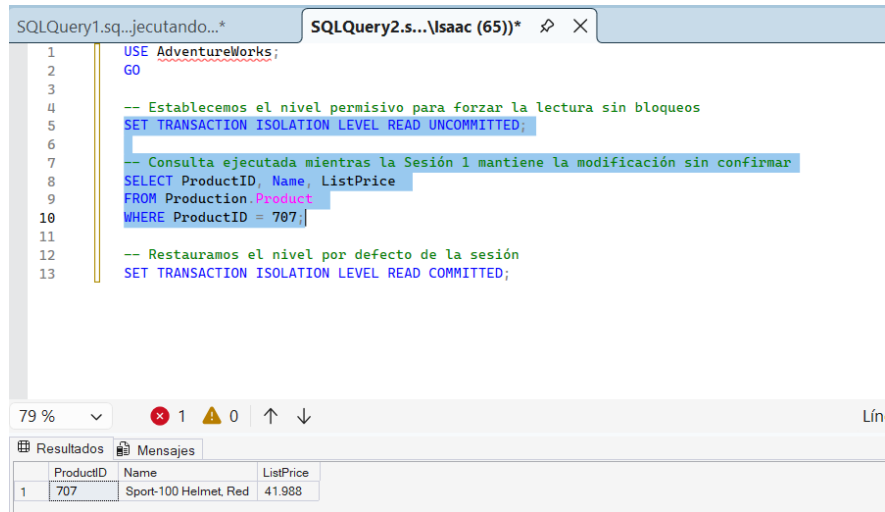
```
SQLQuery1....ecutando... SQLQuery2.sq...\Isaac (65))*
1 USE AdventureWorks2022;
2 GO
3
4 -- Verificamos el valor inicial de control
5 SELECT ProductID, Name, ListPrice
6 FROM Production.Product
7 WHERE ProductID = 707;
8
9 BEGIN TRANSACTION;
10
11 -- Modificamos el precio incrementándolo un 20%
12 UPDATE Production.Product
13 SET ListPrice = ListPrice * 1.20
14 WHERE ProductID = 707;
15
16 -- Simulación de procesamiento transaccional en curso (15 segundos)
17 WAITFOR DELAY '00:00:15';
18
19 -- Revertimos los cambios como si hubiese fallado una regla de negocio
20 ROLLBACK TRANSACTION;
21
22 -- Confirmamos que el precio volvió a su estado original
23 SELECT ProductID, Name, ListPrice
24 FROM Production.Product
25 WHERE ProductID = 707;
```

79 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (17.0 RTM) LAPTO

Sesión 2: La **Sesión 2** devuelve el registro de inmediato sin esperar a que termine la **Sesión 1**, mostrando el valor alterado ($ListPrice * 1.20$).



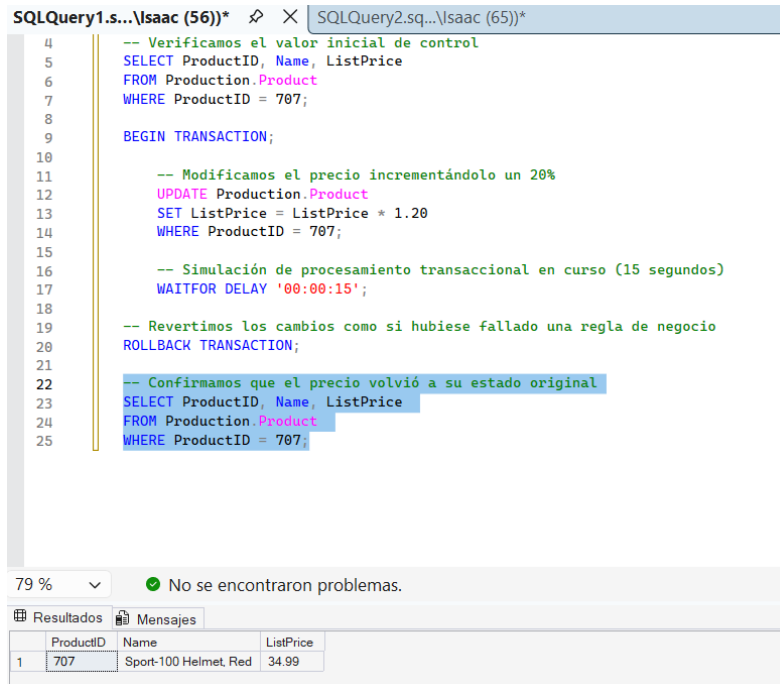
```
SQLQuery1.sq...jecutando...*  SQLQuery2.s...\\Isaac (65))* X
1  USE AdventureWorks;
2  GO
3
4  -- Establecemos el nivel permisivo para forzar la lectura sin bloqueos
5  SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
6
7  -- Consulta ejecutada mientras la Sesión 1 mantiene la modificación sin confirmar
8  SELECT ProductID, Name, ListPrice
9  FROM Production.Product
10 WHERE ProductID = 707;
11
12 -- Restauramos el nivel por defecto de la sesión
13 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

79 % 1 0 Lin

Resultados Mensajes

ProductID	Name	ListPrice
1 707	Sport-100 Helmet, Red	41.988

Una vez que la **Sesión 1** completa el WAITFOR DELAY y ejecuta el ROLLBACK TRANSACTION, su segunda consulta muestra el precio original. Por tanto, la **Sesión 2** operó sobre un valor fantasma que jamás fue persistido en la base de datos.



```
SQLQuery1.s...\\Isaac (56))* X  SQLQuery2.sq...\\Isaac (65))*
4  -- Verificamos el valor inicial de control
5  SELECT ProductID, Name, ListPrice
6  FROM Production.Product
7  WHERE ProductID = 707;
8
9  BEGIN TRANSACTION;
10
11  -- Modificamos el precio incrementándolo un 20%
12  UPDATE Production.Product
13  SET ListPrice = ListPrice * 1.20
14  WHERE ProductID = 707;
15
16  -- Simulación de procesamiento transaccional en curso (15 segundos)
17  WAITFOR DELAY '00:00:15';
18
19  -- Revertimos los cambios como si hubiese fallado una regla de negocio
20  ROLLBACK TRANSACTION;
21
22  -- Confirmamos que el precio volvió a su estado original
23  SELECT ProductID, Name, ListPrice
24  FROM Production.Product
25  WHERE ProductID = 707;
```

79 % No se encontraron problemas.

Resultados Mensajes

ProductID	Name	ListPrice
1 707	Sport-100 Helmet, Red	34.99

¿Porque pasa esto?

La **Sesión 1** adquiere un bloqueo exclusivo (*Exclusive Lock*, **X**) a nivel de fila/clave (KEY) y bloqueos de intención exclusivos (**IX**) sobre la página y la tabla para garantizar que ninguna otra transacción sobrescriba sus cambios.

Al fijar SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED, la **Sesión 2** no solicita bloqueos compartidos (*Shared Locks*, **S**) para leer y omite la comprobación de compatibilidad contra el bloqueo **X** existente, accediendo directamente a la memoria intermedia (*buffer cache*) en estado sucio.

Conclusiones:

Durante la experimentación del fenómeno de Dependencia No Confirmada (Lectura Sucia), se comprobó que el nivel de aislamiento READ UNCOMMITTED suprime la adquisición de bloqueos compartidos (S) por parte de las operaciones de lectura. Esto permite a una transacción secundaria acceder a los datos modificados temporalmente bajo un bloqueo exclusivo (X) sostenido por una transacción no confirmada. Al ejecutarse una instrucción ROLLBACK en la transacción primaria, la información consumida por la transacción secundaria resulta inconsistente e inexistente dentro del historial transaccional válido del RDBMS, vulnerando la consistencia del sistema.

Solucion:

Si cambiamos la instrucción en la **Sesión 2** por el nivel por defecto:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Al estar la Sesión 2 bajo READ COMMITTED, el motor exige adquirir un bloqueo compartido (**S** - *Shared Lock*) sobre esa misma fila para asegurar que solo lee datos confirmados.

La pestaña de la **Sesión 2** se quedará con el ícono de ejecución girando indefinidamente (*Executing query...*).

La consulta queda pausada esperando el evento de espera LCK_M_S (*Lock Manager Shared Lock*).

En el instante exacto en que la **Sesión 1** termina sus 15 segundos y ejecuta ROLLBACK (o COMMIT), se liberan los bloqueos **X**.

En ese milisegundo, la Sesión 2 finalmente adquiere su bloqueo **S**, lee el valor limpio/confirmado (el precio original que restauró el ROLLBACK) y finaliza mostrando el resultado.

Así, el aislamiento previene la lectura sucia sacrificando concurrencia inmediata mediante **latencia por espera de bloqueos**.

```
1 USE AdventureWorks2022;  
2 GO  
3  
4 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
5  
6 -- El hint (READCOMMITTEDLOCK) obliga a adquirir bloqueos S reales, i  
7 SELECT ProductID, Name, ListPrice  
8 FROM Production.Product WITH (READCOMMITTEDLOCK)  
9 WHERE ProductID = 707;  
10  
11 -- Restauramos el nivel por defecto de la sesión  
12 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
13  
14
```

79 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost

```
1 USE AdventureWorks2022;  
2 GO  
3  
4 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
5  
6 -- El hint (READCOMMITTEDLOCK) obliga a adquirir bloqueos S reales  
7 SELECT ProductID, Name, ListPrice  
8 FROM Production.Product WITH (READCOMMITTEDLOCK)  
9 WHERE ProductID = 707;  
10  
11 -- Restauramos el nivel por defecto de la sesión  
12 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
13  
14
```

79 % No se encontraron problemas.

ProductID	Name	ListPrice
707	Sport-100 Helmet, Red	34.99

Resultados Mensajes

Consulta ejecutada correctamente. localhost

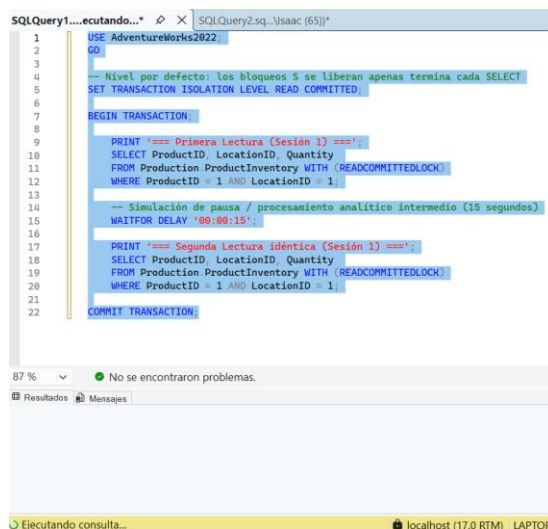
Caso 2: Inconsistent Analysis (Nonrepeatable Read)

La **Lectura No Repetible** (*Non-Repeatable Read* o *Inconsistent Analysis*) ocurre cuando una transacción lee la misma fila dos o más veces dentro de su ámbito de ejecución, pero entre una lectura y otra, una segunda transacción concurrente **modifica o elimina** esa fila y confirma los cambios (COMMIT). Como consecuencia, la primera transacción observa valores distintos para el mismo registro o realiza cálculos de agregación con datos desfasados e inconsistentes entre sí.

- **Ocorre en:** READ UNCOMMITTED y READ COMMITTED. En READ COMMITTED, el motor adquiere un bloqueo compartido (*Shared Lock, S*) para leer la fila, pero **lo libera inmediatamente después de leerla** (a nivel de instrucción/sentencia), sin esperar a que termine la transacción completa. Esto deja la fila desprotegida para que otra sesión la modifique antes de que termine la transacción lectora.
- **Se previene en:** REPEATABLE READ y SERIALIZABLE. En REPEATABLE READ, los bloqueos compartidos adquiridos sobre los registros leídos se retienen **hasta el final de la transacción** (COMMIT o ROLLBACK), impidiendo que otras conexiones alteren o eliminen esos mismos datos.

Para reproducir el escenario expuesto en el texto de referencia, utilizaremos la tabla Production.ProductInventory. La Sesión 1 iniciará una transacción de reporte/auditoría bajo READ COMMITTED para evaluar el inventario de un producto (ProductID = 1 en la ubicación de almacén LocationID = 1), mientras la Sesión 2 altera y confirma el stock en medio de dicho proceso.

Sesión 1:



```
SQLQuery1....ecutando... *  SQLQuery2.sql...Isaac (65)*
1  USE AdventureWorks2022;
2  GO
3
4  -- Nivel por defecto: los bloqueos S se liberan apenas termina cada SELECT
5  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
6
7  BEGIN TRANSACTION;
8
9  PRINT '==== Primera Lectura (Sesión 1) ====';
10 SELECT ProductID, LocationID, Quantity
11 FROM Production.ProductInventory WITH (READCOMMITTEDLOCK)
12 WHERE ProductID = 1 AND LocationID = 1;
13
14 -- Simulación de pausa / procesamiento analítico intermedio (15 segundos)
15 WAITFOR DELAY '00:00:15';
16
17 PRINT '==== Segunda Lectura idéntica (Sesión 1) ====';
18 SELECT ProductID, LocationID, Quantity
19 FROM Production.ProductInventory WITH (READCOMMITTEDLOCK)
20 WHERE ProductID = 1 AND LocationID = 1;
21
22 COMMIT TRANSACTION;
```

87 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (17.0 RTM) LAPTOP...

Sesión 2:

```
SQLQuery1.sq...jecutando...* SQLQuery2.s...\\Isaac (65))*
1 USE AdventureWorks2022;
2 GO
3
4 -- Modificamos y confirmamos inmediatamente mientras la Sesión 1 sigue activa
5 UPDATE Production.ProductInventory
6 SET Quantity = Quantity + 50
7 WHERE ProductID = 1 AND LocationID = 1;
8
9 PRINT 'Actualización completada y confirmada en Sesión 2.';
```

79 % No se encontraron problemas.

Mensajes

(1 fila afectada)
Actualización completada y confirmada en Sesión 2.
Hora de finalización: 2026-09-20T11:25:25.2496929-06:00

Sesión 1:

```
SQLQuery1.s...\\Isaac (56))* SQLQuery2.sq...\\Isaac (65))*
1 USE AdventureWorks2022;
2 GO
3
4 -- Nivel por defecto: los bloqueos S se liberan apenas termina cada SELECT
5 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
6
7 BEGIN TRANSACTION;
8
9 PRINT '=== Primera Lectura (Sesión 1) ===';
10 SELECT ProductID, LocationID, Quantity
11 FROM Production.ProductInventory WITH (READCOMMITTEDLOCK)
12 WHERE ProductID = 1 AND LocationID = 1;
13
14 -- Simulación de pausa / procesamiento analítico intermedio (15 segundos)
15 WAITFOR DELAY '00:00:15';
16
17 PRINT '=== Segunda Lectura idéntica (Sesión 1) ===';
18 SELECT ProductID, LocationID, Quantity
19 FROM Production.ProductInventory WITH (READCOMMITTEDLOCK)
20 WHERE ProductID = 1 AND LocationID = 1;
21
22 COMMIT TRANSACTION;
```

87 % No se encontraron problemas.

Resultados Mensajes

	ProductID	LocationID	Quantity
1	1	1	408

	ProductID	LocationID	Quantity
1	1	1	458

Comportamiento en SSMS:

- La **Sesión 1** ejecuta su primera consulta y obtiene una cantidad inicial (por ejemplo, 408).
- La **Sesión 2** se ejecuta en paralelo y **no se bloquea**: completa el UPDATE de forma instantánea porque la Sesión 1 ya liberó su bloqueo compartido (**S**) al finalizar su primer SELECT.
- Al transcurrir los 15 segundos, la **Sesión 1** despierta y ejecuta exactamente la misma consulta sobre el mismo producto, pero ahora la tabla devuelve un valor distinto (458).
- La transacción de la Sesión 1 experimentó una **Lectura No Repetible**, pues los datos mutaron dentro de su propia frontera transaccional.

Conclusión:

El análisis experimental demostró que el nivel de aislamiento READ COMMITTED garantiza únicamente consistencia a nivel de instrucción, liberando los bloqueos compartidos (S) tan pronto concluye la lectura individual. Esto genera la anomalía de Lectura No Repetible (Inconsistent Analysis) cuando transacciones concurrentes ejecutan operaciones DML de actualización o eliminación sobre los mismos registros antes del término de la transacción lectora. Para asegurar la repetibilidad y consistencia de lectura a lo largo de toda la transacción, se requiere elevar el nivel de aislamiento a REPEATABLE READ, reteniendo los bloqueos compartidos hasta la finalización formal de la transacción.

Solución:

Si la Sesión 1 hubiera usado SET TRANSACTION ISOLATION LEVEL REPEATABLE READ, el bloqueo **S** se habría mantenido activo hasta el COMMIT TRANSACTION final, obligando a la Sesión 2 a esperar a que la Sesión 1 concluyera todas sus lecturas.

Sesión 1:

```
SQLQuery1....ecutando...* X SQLQuery2.sq...\\isaac (65))*
1 USE AdventureWorks2022;
2 GO
3
4 -- Elevamos el nivel de aislamiento para retener los bloqueos S hasta el COMMIT
5 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6
7 BEGIN TRANSACTION;
8
9 -- Primera lectura: SQL Server bloquea la fila con un Shared Lock (S) persistente
10 SELECT ProductID, LocationID, Quantity
11 FROM Production.ProductInventory
12 WHERE ProductID = 1 AND LocationID = 1;
13
14 -- Pausa de 15 segundos
15 WAITFOR DELAY '00:00:15';
16
17 -- Segunda lectura idéntica
18 SELECT ProductID, LocationID, Quantity
19 FROM Production.ProductInventory
20 WHERE ProductID = 1 AND LocationID = 1;
21
22 COMMIT TRANSACTION;
```

87 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (17.0 RTM) LAPTOP-UIBBPMF2

```
SQLQuery1.s...\\isaac (56))* X SQLQuery2.sq...\\isaac (65))*
1 USE AdventureWorks2022;
2 GO
3
4 -- Elevamos el nivel de aislamiento para retener los bloqueos S hasta el COMMIT
5 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6
7 BEGIN TRANSACTION;
8
9 -- Primera lectura: SQL Server bloquea la fila con un Shared Lock (S) persistente
10 SELECT ProductID, LocationID, Quantity
11 FROM Production.ProductInventory
12 WHERE ProductID = 1 AND LocationID = 1;
13
14 -- Pausa de 15 segundos
15 WAITFOR DELAY '00:00:15';
16
17 -- Segunda lectura idéntica
18 SELECT ProductID, LocationID, Quantity
19 FROM Production.ProductInventory
20 WHERE ProductID = 1 AND LocationID = 1;
21
22 COMMIT TRANSACTION;|
```

87 % No se encontraron problemas.

Resultados Mensajes

	ProductID	LocationID	Quantity
1	1	1	558

	ProductID	LocationID	Quantity
1	1	1	558

Consulta ejecutada correctamente. localhost (17.0 RTM) LAPTOP-UIBBPMF2

Sesión 2:

```
SQLQuery1.sq...jecutando...* X SQLQuery2....ecutando...* X
1 USE AdventureWorks2022;
2 GO
3
4 -- Intentamos modificar la misma fila
5 UPDATE Production.ProductInventory
6 SET Quantity = Quantity + 50
7 WHERE ProductID = 1 AND LocationID = 1;
```

87 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (

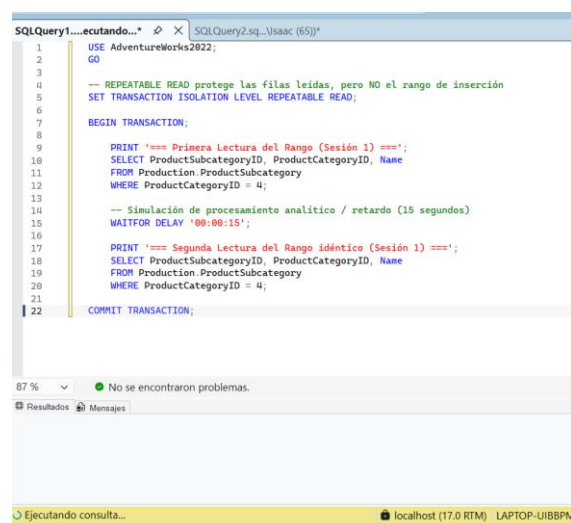
Caso 3: Phantom Reads

La **Lectura Fantasma** (*Phantom Read*) ocurre cuando una transacción ejecuta una consulta basada en un predicado o rango de búsqueda (por ejemplo, una cláusula WHERE que filtra un conjunto de filas), y mientras dicha transacción sigue abierta, una segunda transacción concurrente **inserta** (o elimina) filas nuevas que cumplen con ese mismo criterio de búsqueda y confirma sus cambios (COMMIT). Al volver a ejecutar la misma consulta de rango dentro de la primera transacción, el conjunto de resultados devuelve filas adicionales ("fantasmas") que antes no existían, provocando discrepancias en agregaciones y cálculos analíticos.

- **Ocorre en:** READ UNCOMMITTED, READ COMMITTED y de manera crucial en REPEATABLE READ. En REPEATABLE READ, el motor bloquea las filas individuales que leyó inicialmente para que nadie las modifique, pero **no bloquea el espacio o rango entre ellas**, permitiendo que otra sesión inserte nuevos registros dentro de ese intervalo.
- **Se previene en:** SERIALIZABLE. En este nivel, SQL Server adquiere bloqueos de rango de claves (*Key-Range Locks*), los cuales protegen no solo las filas existentes sino también los huecos (*gaps*) lógicos entre claves en el índice, impidiendo que otra transacción inserte registros que coincidan con el predicado hasta que la primera transacción concluya.

Para reproducir este escenario con la tabla Production.ProductSubcategory, evaluaremos una consulta de rango que filtra subcategorías asociadas a una categoría de producto específica (ProductCategoryID = 4, que corresponde a accesorios).

Sesión 1:



```
SQLQuery1...ejecutando...* X SQLQuery2.sql...\\isaac (65)*
1  USE AdventureWorks2022;
2  GO
3
4  -- REPEATABLE READ protege las filas leídas, pero NO el rango de inserción
5  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6
7  BEGIN TRANSACTION;
8
9  PRINT '==== Primera Lectura del Rango (Sesión 1) ====';
10 SELECT ProductSubcategoryID, ProductCategoryID, Name
11 FROM Production.ProductSubcategory
12 WHERE ProductCategoryID = 4;
13
14 -- Simulación de procesamiento analítico / retardo (15 segundos)
15 WAITFOR DELAY '00:00:15';
16
17 PRINT '==== Segunda Lectura del Rango idéntico (Sesión 1) ====';
18 SELECT ProductSubcategoryID, ProductCategoryID, Name
19 FROM Production.ProductSubcategory
20 WHERE ProductCategoryID = 4;
21
22 COMMIT TRANSACTION;
```

67 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (17.0 RTM) LAPTOP-UI8BPM

Sesión 2:

SQLQuery1.sq...ejecutando...*

SQLQuery2.s...\\Isaac (65))*

```
1  USE AdventureWorks2022;
2  GO
3
4  -- Insertamos un nuevo registro que coincide con el filtro ProductCategoryID = 4
5  INSERT INTO Production.ProductSubcategory (ProductCategoryID, Name, rowguid, ModifiedDate)
6  VALUES (4, 'Accesorio Fantasma Test', NEWID(), GETDATE());
7
8  PRINT 'Fila insertada y confirmada con éxito en Sesión 2.';
```

87 %
No se encontraron problemas.

Mensajes

(1 fila afectada):
Fila insertada y confirmada con éxito en Sesión 2.
Hora de finalización: 2024-09-20T11:46:02.0912000-06:00

87 %
No se encontraron problemas.

Consulta ejecutada correctamente. localhost (17.0 RTM) LAPTOP-UIBBPMF2Vh

Sesión 1:

SQLQuery1.s...\\Isaac (56))*

SQLQuery2.sq...\\Isaac (65))*

87 %
No se encontraron problemas.

Resultados

	ProductSubcategoryID	ProductCategoryID	Name
1	26	4	Bike Racks
2	27	4	Bike Stands
3	28	4	Bottles and Cages
4	29	4	Cleaners
5	30	4	Fenders
6	31	4	Helmets
7	32	4	Hydration Packs
8	33	4	Lights
9	34	4	Locks
10	35	4	Panniers
11	36	4	Pumps
12	37	4	Tires and Tubes

	ProductSubcategoryID	ProductCategoryID	Name
1	26	4	Bike Racks
2	27	4	Bike Stands
3	28	4	Bottles an...
4	29	4	Cleaners
5	30	4	Fenders
6	31	4	Helmets
7	32	4	Hydration ...
8	33	4	Lights
9	34	4	Locks
10	35	4	Panniers
11	36	4	Pumps
12	37	4	Tires and ...
13	38	4	Accesorio ...

Comportamiento en SSMS:

- En la **Sesión 1**, la primera cuadrícula de resultados muestra un número inicial de filas (por ejemplo, 12 subcategorías).
- La **Sesión 2** se ejecuta y concluye de forma inmediata sin quedarse bloqueada, insertando el nuevo registro de manera exitosa.
- Cuando se cumplen los 15 segundos, la **Sesión 1** devuelve su segunda cuadrícula con **13 filas**: la nueva tupla insertada por la Sesión 2 apareció de forma imprevista en medio de la transacción activa de la Sesión 1.
- Esto demuestra que REPEATABLE READ no es suficiente para asegurar la invariabilidad del conjunto de resultados cuando intervienen operaciones INSERT.

Conclusiones:

A través de la experimentación del fenómeno de Lectura Fantasma (Phantom Reads), se verificó que el nivel de aislamiento REPEATABLE READ garantiza la inmutabilidad de los registros leídos individualmente, pero carece de mecanismos para restringir la incorporación de nuevas tuplas que satisfacen el predicado de búsqueda. Al ejecutar una inserción concurrente desde una segunda sesión, el conjunto de resultados de la transacción primaria se expandió de manera inconsistente. La resolución estricta de esta anomalía exige escalar al nivel de aislamiento SERIALIZABLE, el cual implementa bloqueos de rango de claves (Key-Range Locks) para blindar el espacio de claves y garantizar una ejecución serializable equivalente.

Solución:

Para evitar esto, se requiere el nivel SERIALIZABLE, bajo el cual SQL Server utiliza bloqueos de rango de claves como RangeS-S (bloqueo de rango compartido para lectura) que bloquea el intervalo lógico, obligando a cualquier intento de inserción con Rangel-N a esperar a que termine la transacción lectora

Sesión 1:

```
SQLQuery1.sq...jecutando...* SQLQuery2....ecutando...*
1 USE AdventureWorks2022;
2 GO
3
4 PRINT 'Intentando insertar fila en el rango ProductCategoryID = 4...';
5
6 -- Esta instrucción quedará bloqueada en espera hasta que la Sesión 1 haga COMMIT
7 INSERT INTO Production.ProductSubcategory (ProductCategoryID, Name, rowguid, ModifiedDate)
8 VALUES (4, 'Accesorio Fantasma Prevenido', NEWID(), GETDATE());
9
10 PRINT 'Fila insertada exitosamente tras liberarse el bloqueo de rango de La Sesión 1.';
```

87 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (17.0 RTM) LAPTOP-UIBBPMF2\isaac

SQLQuery1.s...\Isaac (56))* SQLQuery2.sq...\Isaac (65))*

```
19 FROM Production.ProductSubcategory
20 WHERE ProductCategoryID = 4;
21
22 COMMIT TRANSACTION;
```

87 % No se encontraron problemas.

Resultados Mensajes

5	30	4	Fenders
6	31	4	Helmets
7	32	4	Hydration Packs
8	33	4	Lights
9	34	4	Locks
10	35	4	Panniers
11	36	4	Pumps
12	37	4	Tires and Tubes

ProductSubcategoryID	ProductCategoryID	Name
1	26	Bike Racks
2	27	Bike Stands
3	28	Bottles and Cages
4	29	Cleaners and Maintenance
5	30	Fenders
6	31	Helmets
7	32	Hydration Packs
8	33	Lights
9	34	Locks
10	35	Panniers
11	36	Pumps
12	37	Tires and Tubes

Sesión 2:

```
SQLQuery1....ecutando...* SQLQuery2.sq...jecutando...*
1 USE AdventureWorks2022;
2 GO
3
4 -- Establecemos SERIALIZABLE para bloquear el rango de claves (huecos e índices)
5 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
6
7 BEGIN TRANSACTION;
8
9 PRINT '=== Primera Lectura del Rango (Sesión 1 - SERIALIZABLE) ===';
10 SELECT ProductSubcategoryID, ProductCategoryID, Name
11 FROM Production.ProductSubcategory
12 WHERE ProductCategoryID = 4;
13
14 -- Ventana de retardo para intentar la inserción concurrente desde la Sesión
15 WAITFOR DELAY '00:00:15';
16
17 PRINT '=== Segunda Lectura del Rango (Sesión 1 - SERIALIZABLE) ===';
18 SELECT ProductSubcategoryID, ProductCategoryID, Name
19 FROM Production.ProductSubcategory
20 WHERE ProductCategoryID = 4;
21
22 COMMIT TRANSACTION;
```

87 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta... localhost (17.0 RTM) LAPTOP-I

En la consulta de la sesión 1 se observa que mantiene las 12 filas en la primera y segunda consulta, es decir, la transacción de la sesión 2 no afectó la consulta de la sesión 1.